

Contexto

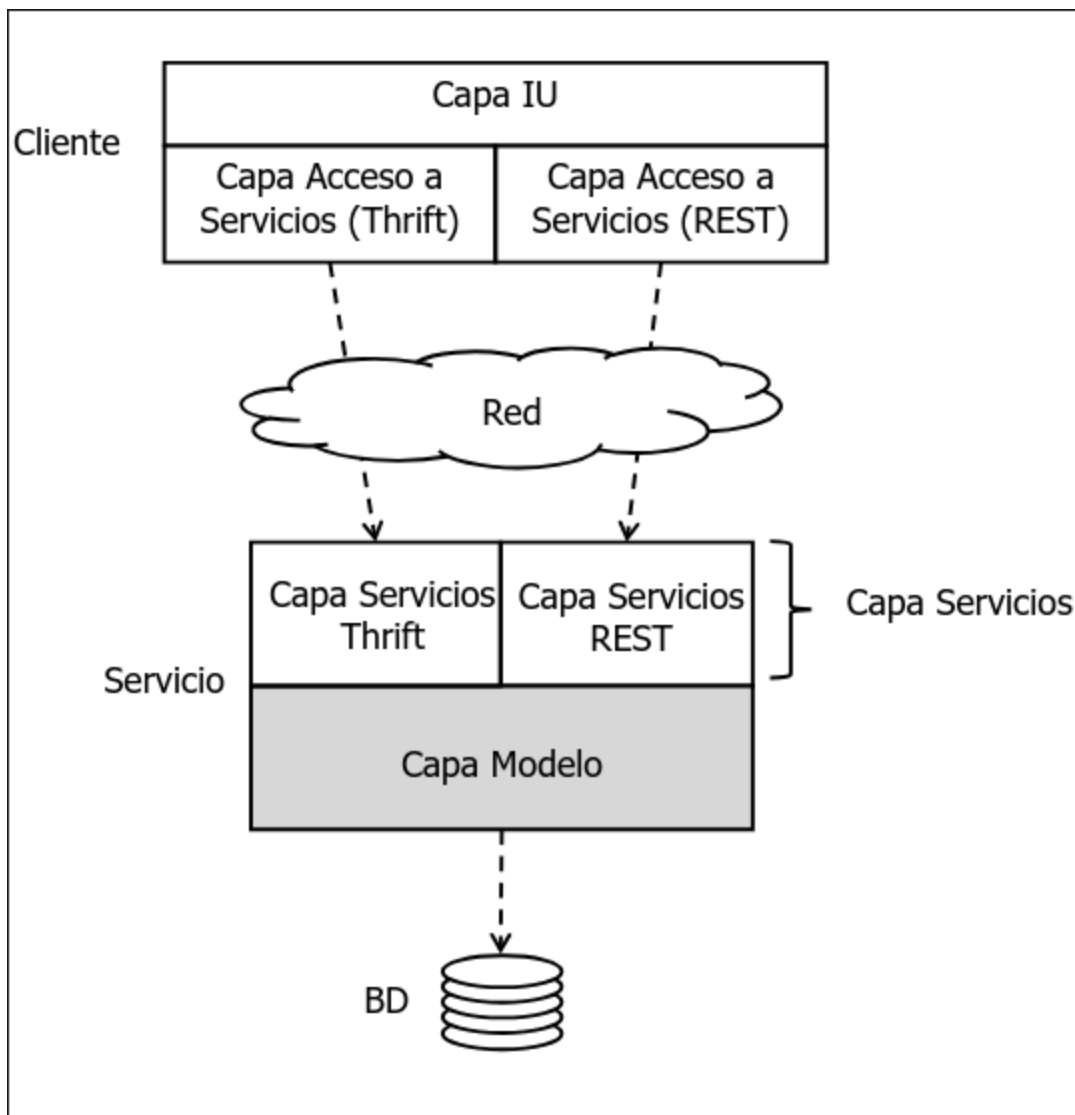
¿Qué es Movies?

Movies corresponde al módulo `ws-moves` de los ejemplos de la asignatura.

Proporciona:

- Una capa Modelo: Ofrece operaciones para gestionar información sobre películas (añadir, eliminar, etc) y comprarlas.
- Dos capas de Servicios, una Thrift y otra RESTful. que delegan en la capa Modelo. Exponen gran parte de la lógica de negocio, mediante una interfaz remota y más adaptada a las necesidades de sus clientes.
- Un sencillo cliente de línea de comandos que puede invocar cualquiera de los servicios a través de una interfaz local común.

Arquitectura general de Movies



¿Qué ofrece la capa Modelo del servicio Movies?

Permite gestionar información sobre **películas**

- Añadir.
- Actualizar.
- Eliminar una película si aún no tiene ninguna compra.
- Buscar una película por su clave.
- Buscar películas por palabras clave de título.

Permite comprar películas

- El usuario compra la película pagando con tarjeta de crédito.
- Cada compra genera una **venta**, que entre otras cosas, contiene una URL de un servicio de Video Streaming que permite visionar la película a veces que desee durante dos días.
- Es posible recuperar la información de la venta a partir de su clave mientras la venta no expire.

Película:

- Identificador (clave numérica), título, duración, descripción, precio, fecha de alta en BD.
- El identificador de la película es numérico y se debe generar automáticamente.

Venta:

- Identificador de la venta, identificador de la película, identificador del usuario, fecha de expiración, número de tarjeta de crédito con la que se compró, precio al que se compró, precio al que se compró la película, URL de streaming, fecha de la venta.
- El identificador de la venta es numérico y se debe generar automáticamente.

Para simplificar el ejemplo, no se modelará la información específica del usuario.

En el contexto de este tema, llamaremos **caso de uso** a cada una de las funcionalidades que ofrece la capa Modelo.

Muchas veces, aunque no siempre, un caso de uso a nivel de modelo corresponde a una funcionalidad concreta que el usuario de la aplicación puede ejecutar

La capa Modelo debe ofrecer una API que:

- **Permita invocar cada caso de uso de manera sencilla** -> facilidad de uso para el desarrollador de la capa superior.
- **Que oculte detalles de implementación** -> es posible modificar la implementación sin que afecte a la capa superior.

Método de desarrollo

1. Modelar las entidades
 2. Definir una API sencilla para invocar los casos de uso
 3. Para cada entidad, crear una abstracción que permita gestionar su persistencia
 4. Implementar los casos de uso
 5. Implementar las pruebas de integración de los casos de uso // [METER ENLACE AL TEMA](#)
- 4

Consideraciones previas

En `ws-javaexamples`: módulos `ws-util` y `ws-movies-model`
`ws-util`:

- Clases reusables.
- Paquete principal: `es.udc.ws.util`
`ws-movies-model`
- Capa Modelo del ejemplo Movies
- Paquete principal: `es-udc.ws.movies.model`

Tratamiento de excepciones en `ws-javaexamples`

Errores lógicos:

- Normalmente representan errores del usuario final
 - Ejemplos: actualizar una película que no existe, añadir una película con información de entrada en formato erróneo, etc.
 - En `ws-javaexamples`, la capa Modelo notifica este tipo de errores lanzando excepciones de tipo "checked" (hijas de `Exception`)
 - Normalmente, este tipo de excepciones se capturan en la interfaz de usuario y se muestran mensajes de error para que el usuario corrija los datos.
 - `ws-util` incluye dos excepciones para dos tipos de errores "lógicos" muy frecuentes en casi cualquier aplicación
 - `InstanceNotFoundException`: indica que se intenta hacer algo sobre un objeto persistente que no existe.
 - `InputValidationException`: indica que se ha proporcionado una información de entrada en formato erróneo.
 - Definidas en `es.udc.ws.util.exceptions`.
- Errores graves:

- Representan un error de infraestructura.
- Ejemplos: la BD está caída, la tabla a la que se intenta acceder no existe, etc.
- Trataremos este tipo de situaciones lanzando `RuntimeException` (unchecked), con la excepción original encapsulada.
- Típicamente este tipo de excepciones se capturan de manera genérica en un único punto y se le indica al usuario que ha ocurrido un error interno

Parámetros de configuración

- Para poder leer parámetros de configuración, `ws-util` proporciona la clase `ConfigurationParametersManager` (paquete `es.udc.ws.util.configuration`)
- Los parámetros se leen de un fichero con nombre `ConfigurationParameters.properties`, que debe estar disponible en el classpath.
- Formato del fichero: ejemplo

```
# MySQL
url=jdbc:mysql ://localhost/example?...
user=example
passowrd=example
```

Paso 1: Modelado de entidades

De la descripción de los casos de uso se deduce que hay que guardar en BD

- La información sobre las películas
- La información sobre las ventas

Por tanto, definimos dos entidades

- `es.udc.ws.movies.model.movie.Movie`
- `es.udc.ws.movies.model.sale.Sale`

Movie	
- movieId: Long - title: String - runtime: short - description: String - price: float - creationDate: LocalDateTime	- saleId: Long - movieId: Long - userId: String - expirationDate: LocalDateTime - creditCardNumber: String - price: float - movieUrl: String - saleDate: LocalDateTime
+ Constructores + métodos get/set	+ Constructores + métodos get/set

NOTA: en una aplicación real las cantidades monetarias se modelan como `java.math.BigDecimal`.

Cada entidad dispone de:

- Atributos privados para modelar el estado
- Métodos públicos `get` para los atributos que se puedan leer
- Métodos públicos `set` para los atributos que se puedan modificar
- La relación entre `Movie` y `Sale` es 1:N
- Una película puede tener asociada múltiples ventas.
- El atributo `movieId` (clave foránea) en `Sale` es el que mantiene la relación.
- El atributo `movieId` (clave foránea) en `Sale` es el que mantiene la relación.
- Atributos de tipo `LocalDateTime`
- No nos interesa almacenar la parte fraccional de un segundo
- En MySQL los vamos a almacenar en columnas de tipo `DATETIME` que no almacenan la parte fraccional.
- Los constructores y los *setters* correspondientes se encargan de truncar la parte fraccional.
 - Si no lo hiciesen, el valor del atributo de la entidad y el valor almacenado en la Base de Datos podrían ser diferentes.

- Esto es especialmente importante para las pruebas automatizadas //////////// METER ENLACE TEMA 4////////// , cuando se comprueba la igualdad de fechas recuperadas de base de datos con fechas generadas en memoria.
- Por ejemplo, como parte de un caso de prueba de un caso de uso que permite añadir una película, podrían ejecutarse, entre otros los siguientes pasos:
 1. Se crea una entidad de tipo `Movie` en memoria.
 2. Se invoca al caso de uso que permite añadir una película. La implementación del caso de uso establece el instante actual como valor de la propiedad `creationDate` y almacena la entidad en Base de Datos.
 3. Se recuperan de Base de Datos los valores almacenados en el paso anterior y se crea con ellos una nueva entidad de tipo `Movie` en memoria.
 4. Se compara la entidad creada en el paso 1, y modificada en el paso 2, con la creada en el paso 3.

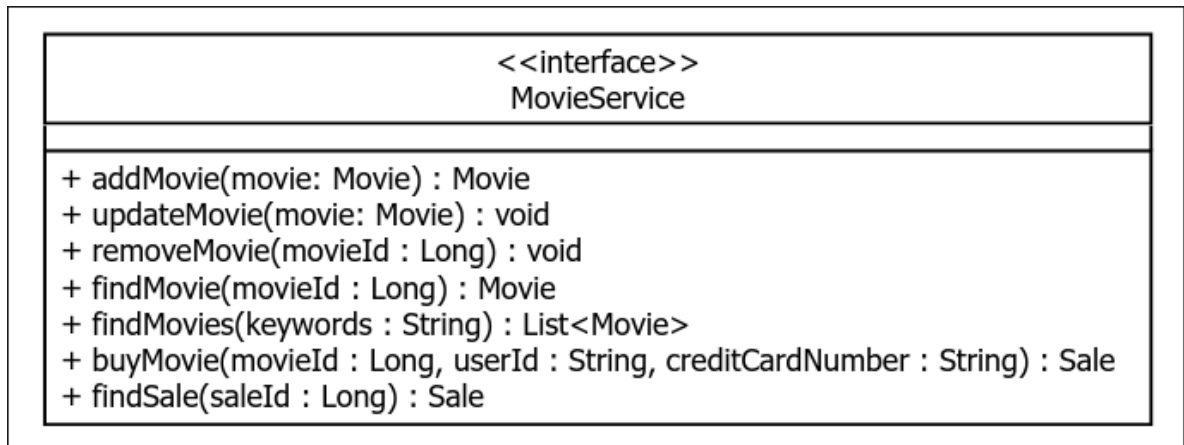
Modelado de entidades

```
public class Movie
{
    ...
    public Movie(Long movieId, String title, short runtime, String
description, float price, LocalDateTime creationDate)
    {
        this(movieId, title, runtime, description, price);
        this.creationDate = (creationDate != null) ?
creationDate.withNano(0) : null;
    }
    ...
    public void setCreationDate(LocalDateTime creationDate)
    {
        this.creationDate = (creationDate != null) ?
creationDate.withNano(0) : null;
    }
    ...
}
```

Paso 2 Definición API modelo

1. Agrupar casos de uso de manera lógica.
 - Distintas personas pueden pensar en distintos criterios para realizar la agrupación
2. Por cada grupo, definir una interfaz.
 - En general, contiene un método por cada caso de uso.

- En cada método, los argumentos corresponden a los datos de entrada del caso de uso y el valor de retorno a los datos de salida.
- Cada una de estas interfaces corresponde a la aplicación del patrón de diseño **Facade**.
- **Por sencillez**, en Movies sólo se ha definido una fachada
 - `es.udc.ws.movies.model.movieservice.MovieService`
 - Convención de nombrado: `XxxService`
 - NOTA: en un caso real, quizás podríamos tener dos fachadas: una con los casos de uso "de administración" (añadir, actualizar, eliminar, etc.) y otra con casos de uso orientados al usuario (e.g buscar, comprar, etc.)



```

public interface MovieService
{
    public Movie addMovie(Movie movie) throws InputValidationException;
    public void updateMovie(Movie movie) throws
InputValidationException, InstanceNotFoundException;
    public void removeMovie(Long movieId) throws
InstanceNotFoundException, MovieNotRemovableException;
    public Movie findMovie(Long movieID) throws
InstanceNotFoundException;
    public List<Movie> findMovies(String keywords);
    public Sale buyMovie(Long movieId, String userId, String
creditCardNumber) throws InstanceNotFoundException,
InputValidationException;
    public Sale findSale(Long saleId) throws InstanceNotFoundException,
SaleExpirationException;
}

```

Paso 3 Gestión de la persistencia

Para guardar las películas y ventas se usan las siguientes tablas. Cada instancia de una entidad se guarda en una fila de la tabla correspondiente.

![[Pasted image 20251022111935.png]]

En general, para implementar los casos de uso se puede necesitar para cada entidad.

- Operaciones básicas: insertar/leer/actualizar/eliminar una instancia
 - CRUD (Create-Read-Update-Delete)
- Operaciones avanzadas, típicamente búsquedas que devuelven una colección de instancias (e.g las películas cuyos tipos contienen un conjunto de palabras clave)

Ejemplo: caso de uso "comprar una película"

- Entrada: `movieId`, `userId`, `creditCardNumber`
- Salida: `Sale`

```
``` Java
```

```
Movie movie = <<Leer de BD los datos de la película cuyo identificador es
"movieId" >>;
LocalDateTime expirationDate = LocalDateTime.now().plusDays(2);
Sale sale = new Sale(movieId, userId, expirationDate, creditCardNumber,
movie.getPrice(), <<getMovieUrl(movieId)>>, LocalDateTime.now());
<<Insertar "sale" en BD>>;
```

Otros casos de uso también pueden necesitar estas mismas operaciones de persistencia (**leer**, **insertar**, etc.) **Por tanto**, para facilitar la implementación de los casos de uso, se necesita una abstracción que permita gestionar la persistencia de cada entidad. **Además**, sería **ideal** que la abstracción ocultase la BD concreta, tipo de BD y tecnología usada para acceder a la BD.

- Si se cambia de BD o tecnología de acceso, no es necesario cambiar la implementación de los casos de uso